



An R package to Compute Least Squares Sparse Principal Components

Giovanni Maria Merola 
British University Vietnam

Abstract

This article illustrates the use of the package *spca*. The computation of sparse principal components is carried out with the LS-SPCA approach. This approach is different from all other SPCA method because it is based on data approximation, rather than components with largest variance. The article includes a brief introduction to LS-SPCA theory and an extended SPC analysis of a small dataset. Most of the options are mentioned and used. Further details can be found in the package documentation.

Keywords: JSS, SPCA, Least squares, projection, uncorrelated, R.

1. Introduction to LS-SPCA

sparse principal components analysis (SPCA) is a variant of principal components analysis (PCA) that aims at computing sparse principal components (sPCs) that are linear combinations of only a few of the features, hence having *sparse* loadings with few nonzero elements.

Most of the existing *conventional* SPCA methods compute sPCs that have maximal variance, using the PCA objective function proposed in Hotelling (1933). There exist a plethora of such methods which, as outlined in the *All sparse PCA models are wrong, but some are useful* series of review articles (Camacho, Smilde, Saccenti, and Westerhuis, 2020, 2021, 2025), produce competing solutions, that are often not close to the original principal components (PCs). Merola and Chen (2019) also shows other drawbacks of these methods.

LS-SPCA was first proposed in Merola (2015) and its projection variant in Merola and Chen (2019). This approach aims to compute sPCs that can be used to best approximate the data, in a least-squares (LS) sense, hence adding sparsity constraints to (Pearson 1901) classic PCA's data approximation objective. It should be noticed that, while Pearson's and Hotelling's objectives give the same solutions in standard PCA, this is not the case when

sparsity constraints are introduced (Merola 2015; Merola and Chen 2019), and this is probably the reason why Camacho *et al.* found that all (conventional) SPCA methods are wrong.

The **spca** package computes LS-SPCA solutions with several options and helper functions to visualize and compare the solutions. Rather than a black-box optimizer type function, the package provides an intuitive UI with transparent options and goodness-of-fit metrics.

This article will be used as extended vignettes in the package, so will illustrate several of the available options and helpers, useful to conduct a thorough SPCA. In the next Section we briefly introduce the theory behind LS-SPCA. In Section ?? we illustrate the package with examples.

1.1. LS-SPCA methodology

The material in this section is taken from Merola (2015) and Merola and Chen (2019) where proofs and further details can be found. Let's establish notation first. X denotes the $n \times p$ data matrix containing n cases on p features centered to mean zero. $S \propto X^\top X$ denotes the covariance matrix. The eigenvectors of S , $v_1, \dots, v_p$ ($\|v_j\| = 1$, $j = 1, \dots, p$ (the PCs loadings)) are ordered by decreasing corresponding eigenvalues, $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$. The same convention is used for all eigenvectors (including generalized ones).

The sparse loadings are the vectors $a_j = (a_{j1}, \dots, a_{jr}, r \leq p$ with unit norm $a_j^\top a_j = 1$, each of which has only $c_j \leq p$ non-zero elements. The quantity c_j is called the *cardinality* of a_j . The sPCs are then equal to $t_j = X a_j$.

A dot above a symbol denote the operation of retaining the elements of a vector or the columns of a matrix corresponding to a set of indices, ind_j . In R syntax, we write $\dot{a}_j = a_j[ind_j]$ to indicate the vector containing only the c_j non-zero loadings in a_j , so that $t_j = \dot{X}_j \dot{a}_j$.

LS-SPCA objective

LS-SPCA objective is obtained by adding sparsity constraints to the standard least squares PCA objective of approximating the data. This is equivalent to maximizing the variance explained (VEXP) by each PC. The objective can be expressed in recursive terms as

$$\min_{a_j} \|X - t_j b^\top\|^2, \quad \text{subject to} \quad \text{card}(a_j) < p, \quad a_j^\top S a_i = 0, \quad i < j; j = 1, \dots, r. \quad (1)$$

LS-SPCA estimation

There exist three variants of LS-SPCA, which have different computational complexity and degree of agreement with standard PCA solutions

Uncorrelated LS-SPCA (uSPCA). The optimal sparse loadings for objective 1, $\dot{a}_j$, are the generalized eigenvectors corresponding to the largest eigenvalue, λ_j , solution to

$$C_j \dot{S}^\top \dot{S} \dot{a}_j = \lambda_j \dot{X}_j^\top \dot{X}_j \dot{a}_j, \quad (2)$$

where $\dot{S} = S[ind_j]$ and the matrices C_j enforce the orthogonality constraints ($C_1 = I$).

The orthogonality constraints require that the cardinality of the solution loadings satisfy $c_j \geq j$. Furthermore, the solutions can be computationally demanding and unstable.

Correlated LS-SPCA (cSPCA). Approximated but simpler to compute solutions can be obtained from the residual matrices orthogonal to the previously computed sPCs. Let

$$Q_j = \left(I_p - \frac{t_{j-1}t_{j-1}^\top}{t_{j-1}^\top t_{j-1}} \right) Q_{j-1}, \quad j = 2, 3, \dots, r, \quad (3)$$

with $Q_1 = X$. In SPCA literature the residual matrices Q_j are called *deflated* matrices. Note that the use of the deflated matrices is a direct consequence of the Courant–Fischer min–max theorem and is typically used in the power method for computing the eigen-decomposition. Then, the cSPCA solution loadings are obtained as the generalized eigenvectors corresponding to the largest eigenvalue:

$$\dot{X}_j^\top Q_j Q_j^\top \dot{X}_j \dot{\mathbf{a}}_j = \mu_j \dot{X}_j^\top \dot{X}_j \dot{\mathbf{a}}_j. \quad (4)$$

The first cSPCA solution is equal to the first uSPCA solution but, in general, the following solutions are not equal to the corresponding uSPCA solutions and, typically show very mild correlation with one another, with increasing values for sPCs of higher order. However, their cardinality is not constrained by the uncorrelatedness requirements and, in most cases, it is lower than their order.

Projection SPCA (pSPCA) A computationally simpler method is based on computing the sparse loadings by requiring that the sPCs best approximate the first PCs of the residual matrices Q_j . Let z_j be the j th residual PC, the pSPCA objective function reduces to

$$\max_{a_j} \text{textcor}(t_j, z_j)^2, \quad \text{subject to } \text{card}(a_j) < p. \quad (5)$$

It is well known that the optimal solution to this objective is linear regression, which projects the residual PCs onto a subset of the features. Therefore, in ‘pSPCA’, the sparse loadings are the coefficients, standardized to unit norm, of the regression of the residual PCs onto subsets of the x -variables. That is

$$\dot{\mathbf{a}}_j \propto (\dot{X}_j^\top \dot{X}_j)^{-1} \dot{X}_j^\top z_j, \quad (6)$$

In general, the pSPCA loadings are close to the cSPCA ones but show higher mutual correlation, still of small magnitude. This approach ensures best approximation sPC to PC and is computationally simpler. Furthermore, pSPCA provide a very efficient approach to variable selection.

1.2. Variable selection

Variable selection is notoriously a computationally demanding (NP-hard) task that requires greedy algorithms for its solution.

For uSPCA and cSPCA we implemented *intensive* forward selection algorithms based on the actual variable explained that terminate after a minimal proportion of the cumulative variance explained (CVEXP) of the corresponding number of PCs is reached. These are computationally expensive, especially for large matrices, because they require to repeatedly compute generalized eigen-decompositions.

We also implemented efficient regression based standard forward, backward, and forward step-wise variable selection algorithms. These algorithms always use the computationally cheap R^2 to select or eliminate variables. For maximum sPC-PC correlation objective the current R^2

should be used as stopping rule. For maximum VEXP objective the current CVEXP should be used. In this case, at every search step, the current CVEXP is computed to evaluate the stopping rule, so they can still be computationally expensive. Forward selection is advised when p is large. Different types of variable selection are chosen via the argument `var_selection = c("fwd", "bkw", "step")` and the stop criterion via the argument `stop_criterion = c("r2", "cvexp")`. The variance explained based intensive selection is selected by setting `intensive = TRUE`.

2. Computational details

As a general approach, in **spca** we tried to make the computation burden low, with an eye on large matrices. Linear algebra and matrix computations are handled by the C++ back-end. This is used also to prevent deep copies, typically made by R functions. The exported C++ functions are called from R wrappers.

The main function `spca()` is used to compute LSSPCA solutions of all types with all options. Different back-ends manage “fat” matrices ($n < p$) and “tall” matrices. The tall back-end operates on the covariance matrix S , while the fat back-end operates on the row-space of the data matrix, using a *reverse-SVD* approach to compute eigen-decompositions and dedicated variable selection algorithms to compute LS-SPCA solutions. Which back-end is used is decided automatically, based on whether the data matrix aspect-ratio is less of one or not, if `fat_matrix = NULL`, or it can be forced by setting `fat_matrix = TRUE/FALSE`.

The major expedient that we use is to do the potentially demanding $O(np^2)$ multiplication to compute the covariance matrix $S = X^T X$ and the $O(p^3)$ multiplication SS only once at the beginning. Then deflations are done with cheap rank-1 updates instead of recomputing the matrices as products of the deflated data matrix. The computational savings can be substantial when p is large, as they reduce the cost from , roughly, $O(p^3)$ to $O(p^2)$.

Since, apart from the initial eigen-decomposition, only the leading eigen-pair of a matrix is needed, these can optionally be computed with the power method. The power method option can be triggered by setting `PML = TRUE` for loading computation and `PMVS = TRUE` for intensive variable selection computation. Using the power method reduces the computational cost for a $p \times p$ matrix from approximately $O(p^3)$ to approximately $O(Kp^2)$, where K is the number of iterations required for convergence. The convergence can be controlled with the `maxiterPM...` and `epsPM...` for each set of computations.

Regression based variable selection algorithm is simplified by considering that the regressed variable is an eigen-vector of the regressors and uses rank-1 updates for computing current values of selected stopping criterion.

Similar considerations are valid for the fat back-end engine.

3. Application and illustrations

The **spca** UI provides the following functions:

fitting functions: `spca()`, `pca()`

methods: `print()`, `plot()`, `summary()`

helper functions: `show_contributions_spca()`, `compare_spca()`,
`aggregate_by_group()`, `new_spca()`, `change_loadings_sign_spca()`,
`spca_screepplot()`, `wachter_qqplot()`

3.1. An example of LS-SPC Analysis

To illustrate a typical SPCA work-flow we use the ‘holzinger’ dataset included in the package. This is the small classic Holzinger-Swineford dataset with 145 cases on 12 variables. Data are loaded as usual as

```
R> data(holzinger)
R> dim(holzinger)
```

```
[1] 145  12
```

3.2. Decide the number of components using `pca()`

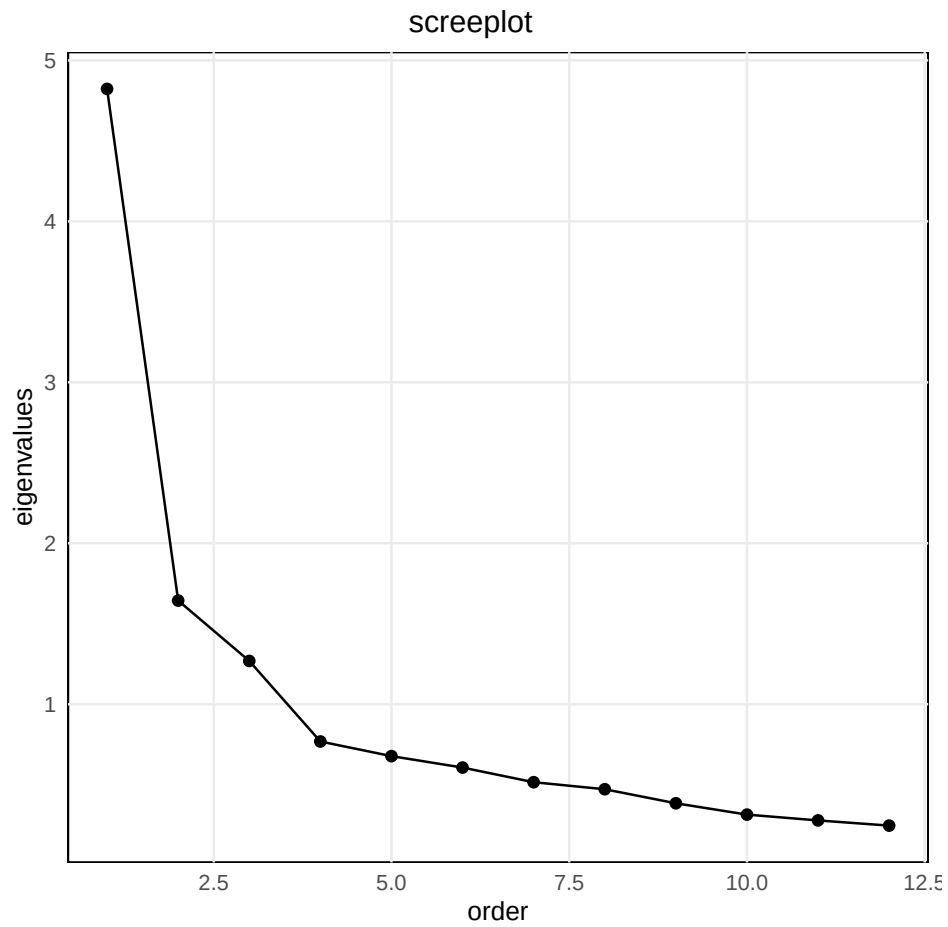
We usually begin the analysis by inspecting PCA results. This can be done by using the function

```
pca(M, ncomps = NULL, centerdata = FALSE, scaledata = FALSE,
     screepplot = TRUE, wachter = TRUE, nrow_data = NULL, neigen_toplot = NULL),
```

which returns an object of class ‘`spca`’. `M` can be either the data matrix or the covariance or correlation matrix (from now on, we use covariance matrix for both cases). In the former case PC scores will not be computed.

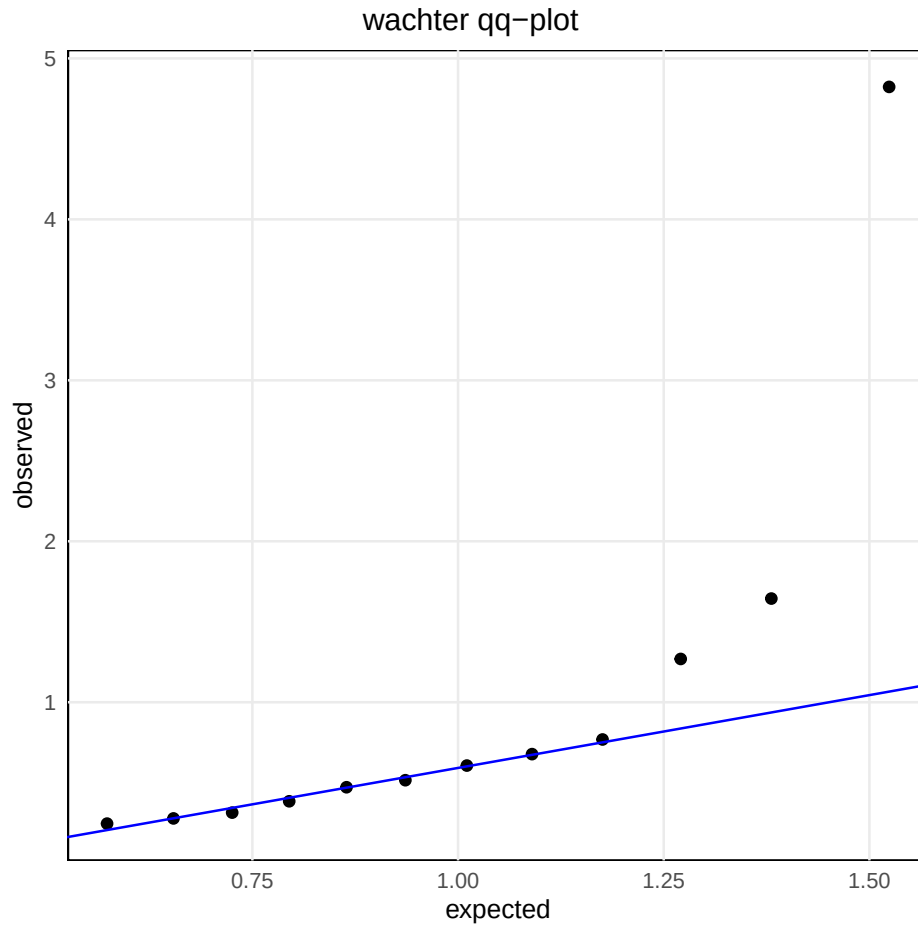
A classic screepplot or a Wachter qq-plot (which uses the quantiles of the Marchenko-Pastur distribution) can be produced. Since the latter plot considers the data aspect-ratio, `nrow_data` is needed if `M` is a covariance matrix. The commands below, produce the PCA output with all components (‘`ncomps = NULL`’).

```
R> ho_pca = pca(holzinger, screepplot = TRUE, wachter = TRUE)
```



The screeplot suggests to retain 4 components whereas the qq-plot indicates 3. We can check this by fitting a line to the first $(p - 4)$ eigenvalues in the plot.

```
R> wachter_qqplot(ho_pca$vexpPC*12, p = ncol(holzinger),
+                 n = nrow(holzinger), n_fitline = -3)
```



We settle for 4 components for caution.

3.3. cSPCA fit

We start with the default values: `M` is the data matrix, `alpha` is the proportion of objective to reach, `CVEXP` in this case, as `stop_criterion` will take the first element of the vector. `method = "c"` indicates that the loadings will be computed with cSPCA and `var_selection = c("fwd", "bkw", "step")` that the variables will be selected with forward selection based on R^2 , as indicated by `intensive = FALSE`.

```
R> ho_spcadef = spca(M = holzinger,
+                    alpha = 0.95,
+                    ncomps = 4,
+                    method = "c",
+                    var_selection = c("fwd", "bkw", "step"),
+                    stop_criterion = c("cvexp", "r2")
+                    )
```

3.4. print

`print()` is the default method. By default it shows the percentage contribution (when the

parameter `contributions = TRUE`) with the variables that do not load any sPC removed (when `only_nonzero = TRUE`).

```
R> ho_spcadef
```

	sPC1	sPC2	sPC3	sPC4
visual	11.9%			43.9%
cubes			31.4%	-21.3%
flags	14.2%		23.0%	
paragraph		21.6%		
sentence	19.6%		-29.7%	
wordm		22.3%		
addition	12.2%	-27.5%	-15.9%	-10.6%
counting		-28.6%		
straight	12.3%			
deduct	13.7%			-24.3%
series	16.1%			
	-----	-----	-----	-----
Cvexp	38.6%	51.5%	61.4%	67.5%

The `print()` function takes the following arguments

```
print.sPCA = function (x, cols = NULL, only.nonzero = TRUE,
  contributions = TRUE,
    digits = 3, thresh = 0.001, rtn = FALSE, components = NULL,
    ...)
```

`cols` which columns to print; by default all columns are shown. If an integer is passed, columns `1:cols` are shown.

Non-zero contributions can be visualized individually by calling `show_contributions_sPCA`

```
R> print("Loadings")
```

```
[1] "Loadings"
```

```
R> ho_spcadef$loadlist[1:2]
```

```
$sPC1
```

visual	flags	sentence	addition	straight	deduct	series
0.4205005	0.5099249	0.3209397	0.3689273	0.3561735	0.3188142	0.3091023

```
$sPC2
```

paragraph	wordm	addition	counting
0.5672593	-0.4418786	0.5461974	-0.4296842

```
R> show_contributions_sPCA(ho_spcadef, cols = 1:2)
```



```
$sPC1
      visual      flags sentence addition straight deduct series
0.1614588 0.1957949 0.1232306 0.1416564 0.1367593 0.1224145 0.1186854
```

```
$sPC2
 paragraph      wordm      addition      counting
0.2857701 -0.2226067 0.2751597 -0.2164634
```

```
$sPC3
      cubes      flags      sentence      addition
0.3135691 -0.2965215 0.2304145 -0.1594949
```

```
$sPC4
      visual      cubes      addition      deduct
0.4387985 -0.2131231 -0.2425778 -0.1055006
```

3.5. summary

Summaries of the `sPCA` output can be produced by calling `summary`

```
R> summary(ho_spcadef)
```

	sPC1	sPC2	sPC3	sPC4
Vexp	38.6%	12.9%	9.9%	6.1%
Cvexp	38.6%	51.5%	61.4%	67.5%
Rvexp	96.0%	94.5%	93.5%	95.3%
Rcvexp	96.0%	95.6%	95.3%	95.3%
Card	7	4	4	4

The table shows the following metrics

- Vexp The percentage variance explained.
- Cvexp The percentage cumulative variance explained.
- Rvexp The variance explained relative to the corresponding PC.
- Rcvexp The cumulative variance explained relative to the corresponding PCs.
- Card The number of non zero loadings.

The cardinality of the sPCs is significantly smaller than the number of features (12) and all sPCs explain more than 95% of the VEXP of the corresponding PC. The same is true for CVEXP, as required.

Even though the sPCs were computed with cSPCA, so allowing for correlated sPCs, these correlations are quite small (note how sPC4 shows higher correlation).

```
R> round(ho_spcadef$corComp, 2)
```

	sPC1	sPC2	sPC3	sPC4
sPC1	1.00	0.01	-0.03	-0.02
sPC2	0.01	1.00	0.08	0.10
sPC3	-0.03	0.08	1.00	-0.06
sPC4	-0.02	0.10	-0.06	1.00

The correlations between sPCs and PCs, shown below, indicates that while the first three sPCs are highly correlated with the corresponding PCs, the fourth is not. Thus confirming that number of important PCs is three, while the fourth is explaining a substantial portion of noise.

```
R> round(diag(cor(ho_spcadef$scores, ho_pca$scores[, 1:4])), 2)
```

```
[1] 0.98 0.95 0.92 -0.76
```

sPC4 is negatively correlated with PC4. The sign of the loadings can be easily changed (together with all other internally stored elements) as

```
R> ho_spcadef = change_loadings_sign_spca(ho_spcadef, 4)
R> round(diag(cor(ho_spcadef$scores, ho_pca$scores[, 1:4])), 2)
```

```
[1] 0.98 0.95 0.92 0.76
```

3.6. plots

The function `plot` has several arguments. The arguments grouped in `controls` are **ggplot2** parameters and are included for convenience. The plots can be changed manually after saving the it with `return_plot = TRUE`.

```
plot.spca = function (
  x,
  nplot = NULL,
  plot_type = c("bars", "circular", "heatmap"),
  contributions = TRUE,
  only_nonzero = TRUE,
  pc_loadings = NULL,
  variable_groups = NULL,
  plot_title = NULL,
  return_plot = FALSE,
  produce_plot = TRUE,
  controls = list(
    color_scale = c("ggplot", "cbb", "printsafe", "bw"),
    variable_names = NULL,
    legend_position = c("none", "bottom", "right", "top", "left"),
    grid_type = c("horizontal", "full", "none"),
    facet_labels = NULL,
```

```

legend_title = NULL,
x_axis_lab = "variables",
adjust_labels_circ = NULL,
flip_heatmap = FALSE,
heatmap_color_range = c("values", "unit")),
...)

```

Most arguments' name should be self-explanatory. If a set of PC loadings is passed to `PCloadings`, the plots will compare them to the sPCs loadings (not implemented for circular barplots). If a vector or a factor indicating groups of variables is passed to `variable_groups`, the plots will show these groups. `plot()` produces three types of plots; the default settings produce a classic barplot

```
R> plot(ho_spcadef)
```

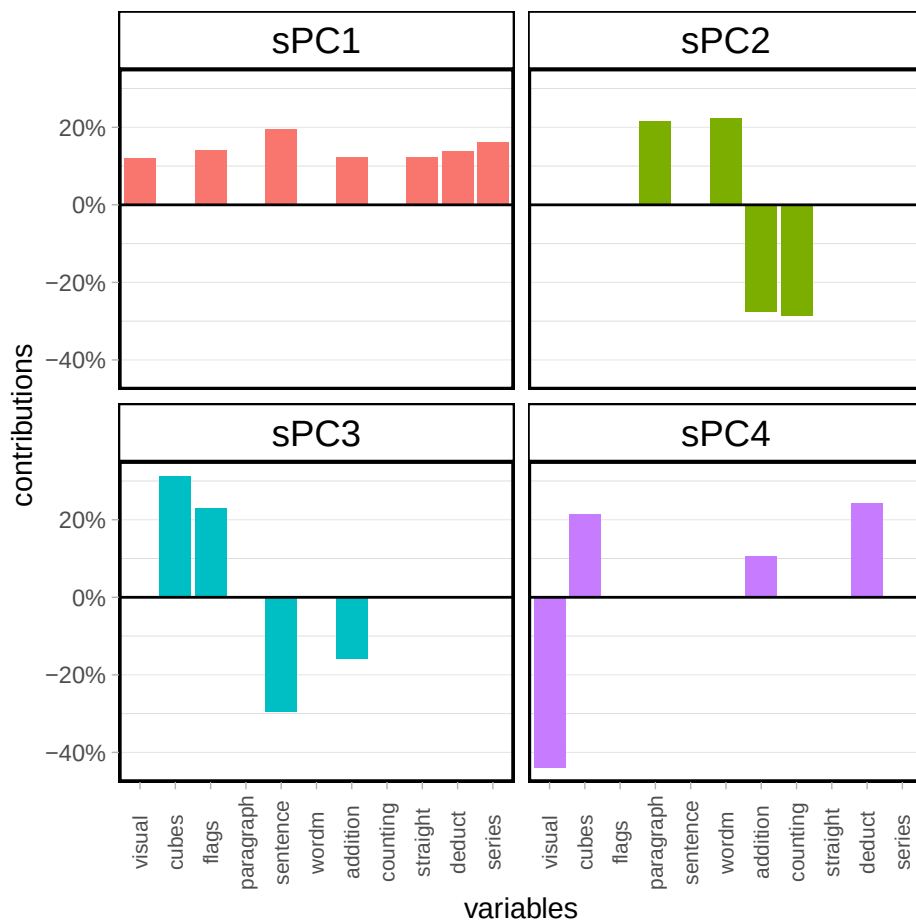
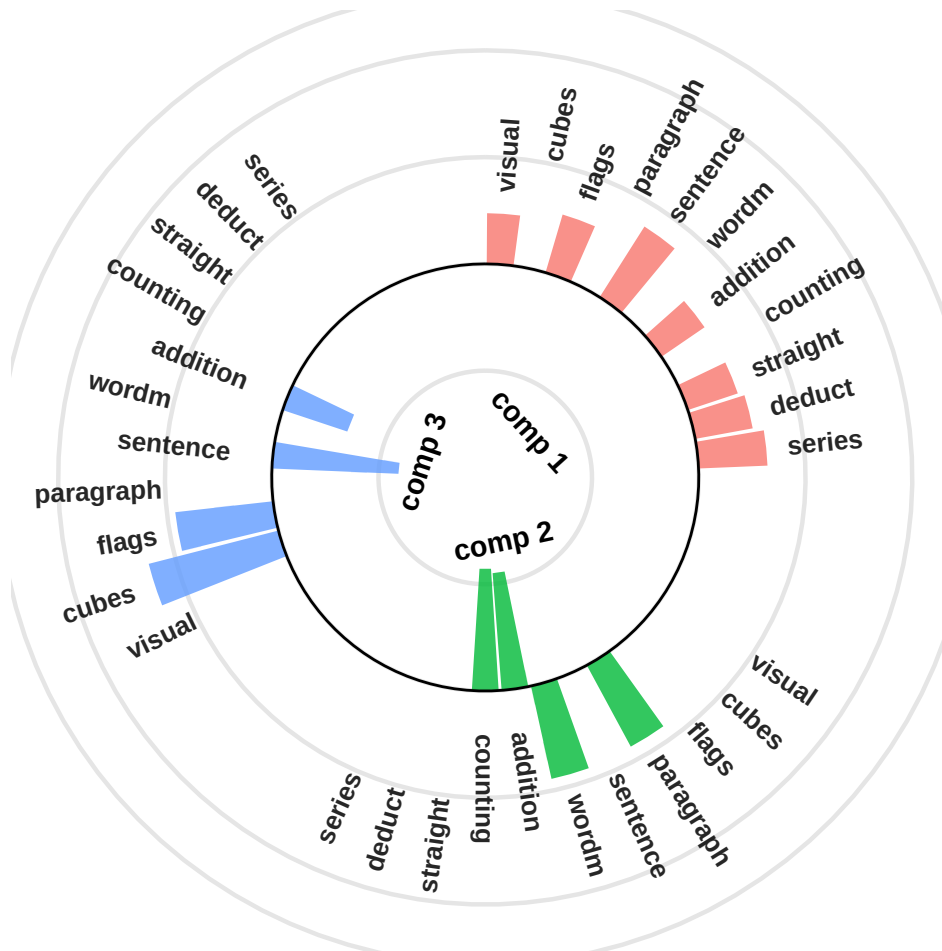


Figure 1: bar plots of the contributions of each sPC.

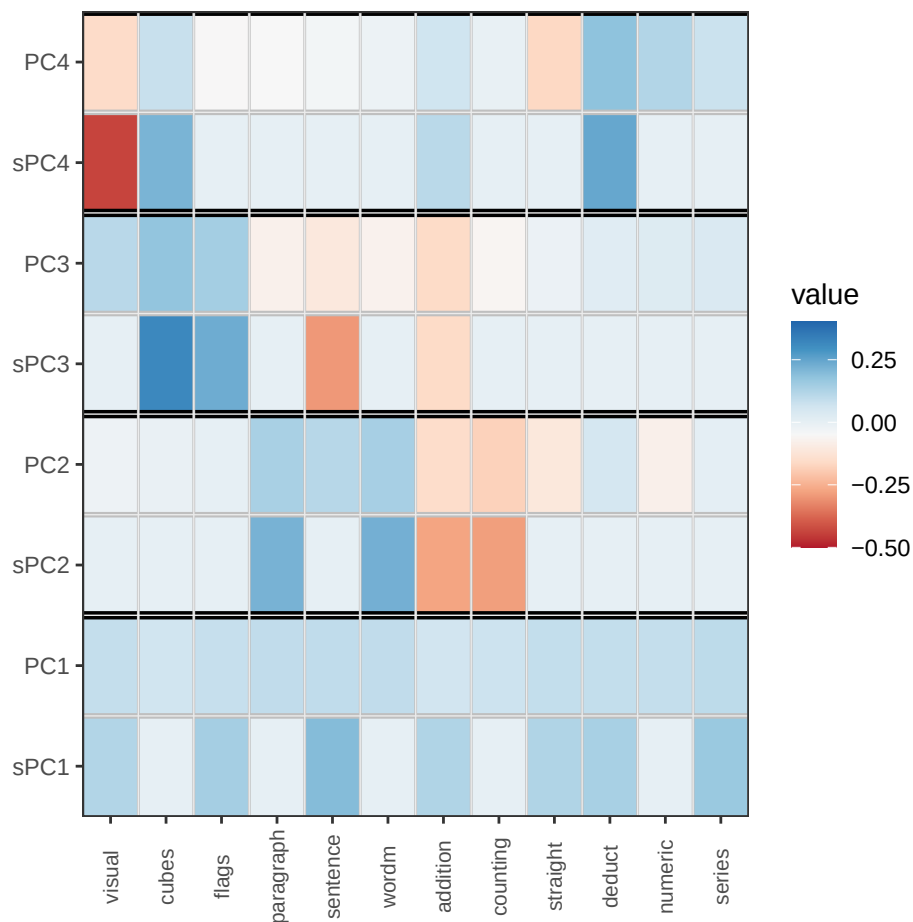
A more compact circular plot is produced by selecting `plot_type = "c"`

```
R> plot(ho_spcadef, nplot = 3, plot_type = "c")
```



SPCA contributions can be compared with the PCA contributions with barplots or heat maps, for example as

```
R> plot(ho_spcadef, pc_loadings = ho_pca$contributions, plot_type= "h")
```



3.7. utilities

There are other utilities available.

compare solutions

We fit a new model requiring that the sPCs have at least 0.95 correlation with the residual PCs, using pSPCA to compute the loadings and backward elimination to select the variables.

```
R> ho_pspca = spca(holzinger, ncomps = 4, alpha = 0.95, method = "p",
+                 stop_criterion = "r2", var_selection = "b")
R> round(diag(cor(ho_pspca$scores, ho_pca$scores[, 1:4])), 2)
```

```
[1] 0.98 0.99 0.98 -0.94
```

```
R> ho_pspcca = change_loadings_sign_spca(ho_pspca, 4)
```

cacca

Numerical and visual comparisons with `ho_spcadef` can be obtained calling `compare_spca()`. We add methods names to be used in the plot but keep the default argument `col_short_names = TRUE` to keep the tables compact.

```
R> ho_compare = compare_sPCA(list(ho_spcadef, ho_pspca), plot_loadings = TRUE,
+                               print_loadings = TRUE, color_scale = "c",
+                               methods_names = c("cSPCA", "pSPCA")
+                               )
```

```
[1] "Percentage Contributions"
```

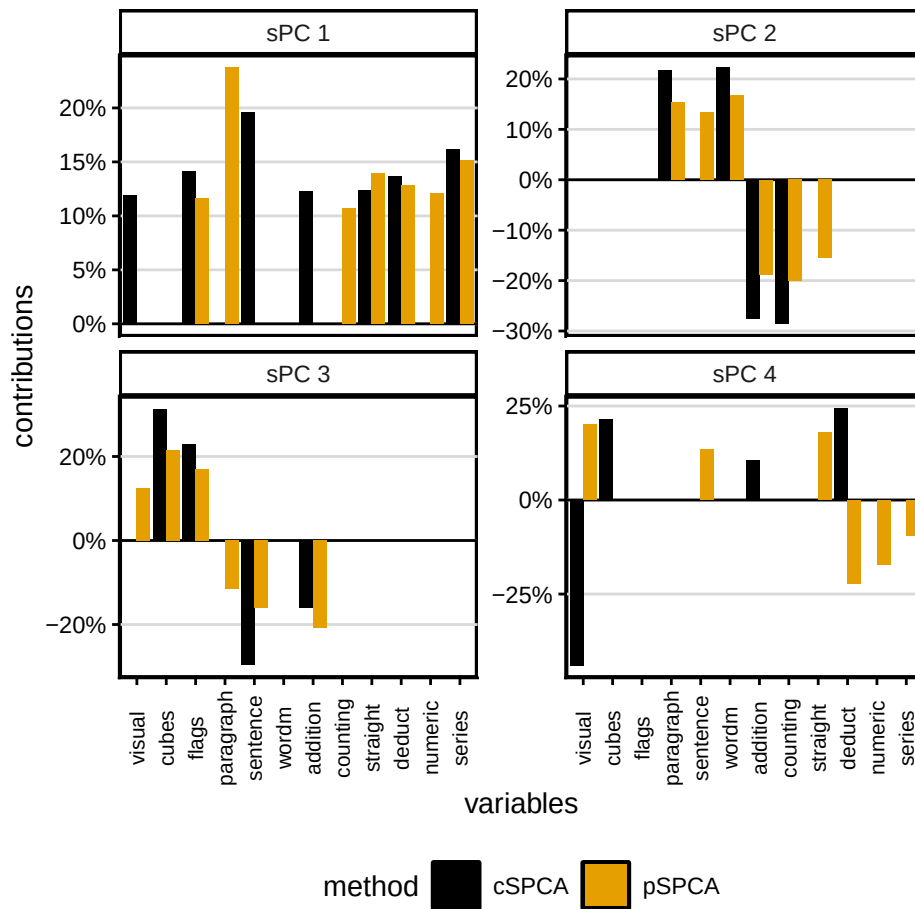
	C1.M1	C1.M2	C2.M1	C2.M2	C3.M1	C3.M2	C4.M1	C4.M2
visual	11.9					12.5	-43.9	20
cubes					31.4	21.6	21.3	
flags	14.2	11.6			23	17.1		
paragraph		23.7	21.6	15.4		-11.6		
sentence	19.6			13.5	-29.7	-16.2		13.4
wordm			22.3	16.7				
addition	12.2		-27.5	-18.8	-15.9	-20.9	10.6	
counting		10.7	-28.6	-20.1				
straight	12.3	13.9		-15.6				17.9
deduct	13.7	12.8					24.3	-22.2
numeric		12.1						-17.1
series	16.1	15.2						-9.4

```
[1] Summary statistics
```

	C1.M1	C1.M2	C2.M1	C2.M2	C3.M1	C3.M2	C4.M1	C4.M2
Vexp	38.6%	38.6%	12.9%	13.5%	9.9%	10.4%	6.1%	6.4%
Cvexp	38.6%	38.6%	51.5%	52.1%	61.4%	62.5%	67.5%	68.8%
Rvexp	96.0%	96.0%	94.5%	98.5%	93.5%	97.9%	95.3%	99.8%
Rcvexp	96.0%	96.0%	95.6%	96.7%	95.3%	96.9%	95.3%	97.1%
Card	7	7	4	6	4	6	4	6
Min cont	11.9%	10.7%	21.6%	13.5%	15.9%	11.6%	10.6%	9.4%

```
R> round(cor(ho_spcadef$scores, ho_pspca$scores), 2)
```

	sPC1	sPC2	sPC3	sPC4
sPC1	0.96	0.01	-0.02	0.07
sPC2	0.04	0.96	0.12	0.01
sPC3	0.04	-0.05	0.94	-0.12
sPC4	0.09	-0.01	-0.18	-0.65



The pSPCA loadings have from the second sPC have larger cardinality and larger VEXP. This is to be expected because VEXP cannot be lower than the R^2 optimized. The contributions for the first three sets of loadings are not very dissimilar, considering the the different methods used. The sPCs scores are also are also highly mutually correlated. As noted before, the fourth sPC is unstable.

Feature groups

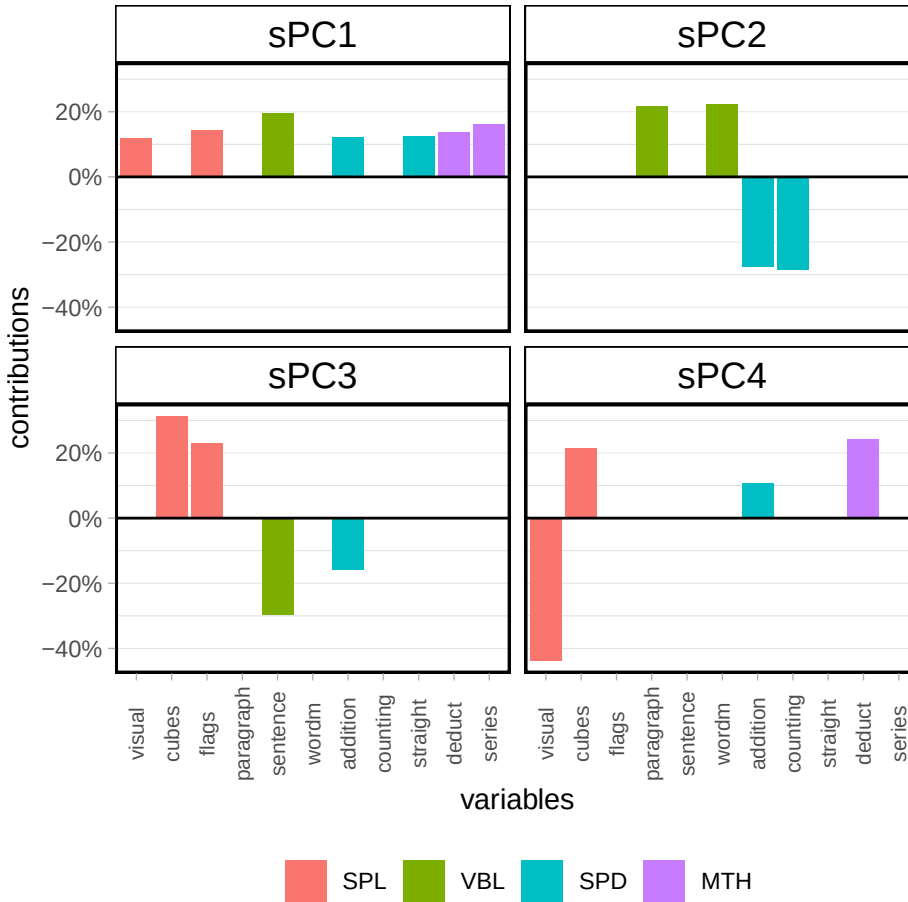
In some cases, typically for questionnaire data, features belong to different groups. For our data the 12 features are grouped in 4 scales.

The function `aggregate_by_group` uses the index vector to sum the loadings or contributions in each scale; adding the vector to `plot()` returns plots with scales filled with different colors

```
R> aggregate_by_group(ho_spcadef, groups = holzinger_scales)
```

```
[1] "percentage contributions"
      sPC1  sPC2  sPC3  sPC4
SPL  26.0%  0.0%  54.4% -22.6%
VBL  19.6%  43.9% -29.7%  0.0%
SPD  24.6% -56.1% -15.9%  10.6%
MTH  29.8%  0.0%  0.0%  24.3%
```

```
R> plot(ho_spcadef, variable_groups = holzinger_scales)
```



Create an spca object

spca functions can be applied to results from other analysis by casting them as an **spca** object using the **new_spca** function. It requires a set of loadings and the covariance (or correlation) matrix needs

```
R> A = cbind(ho_spcadef$loadings[, 1], ho_pspca$loadings[, 2])
R> ho_r = cor(holzinger)
R> ho_spcahyb = new_spca(A, ho_r, method_name = "hybrid")
R> is.spca(ho_spcahyb)
```

```
[1] TRUE
```

References

Camacho J, Smilde A, Saccenti E, Westerhuis J (2020). "All sparse PCA models are wrong, but some are useful. Part I: Computation of scores, residuals and explained variance."

- Chemometrics and Intelligent Laboratory Systems*, **196**, 103907. ISSN 0169-7439. doi: <https://doi.org/10.1016/j.chemolab.2019.103907>.
- Camacho J, Smilde A, Saccenti E, Westerhuis J, Bro R (2021). “All sparse PCA models are wrong, but some are useful. Part II: Limitations and problems of deflation.” *Chemometrics and Intelligent Laboratory Systems*, **208**, 104212.
- Camacho J, Smilde A, Saccenti E, Westerhuis J, Bro R (2025). “All sparse PCA models are wrong, but some are useful. Part III: Model interpretation.” *Chemometrics and Intelligent Laboratory Systems*, **266**, 105498. ISSN 0169-7439. doi: <https://doi.org/10.1016/j.chemolab.2025.105498>.
- Hotelling H (1933). “Analysis of a Complex of Statistical Variables with Principal Components.” *Journal of Educational Psychology*, **24**, 498–520.
- Merola GM (2015). “Least Squares Sparse Principal Component Analysis: a Backward Elimination approach to attain large loadings.” *Australia & New Zealand Journal of Statistics*, **57**, 391–429.
- Merola GM, Chen G (2019). “Projection sparse principal component analysis: An efficient least squares method.” *Journal of Multivariate Analysis*, **173**, 366 – 382. URL <http://www.sciencedirect.com/science/article/pii/S0047259X18305463>.
- Pearson K (1901). “On lines and planes of closest fit to systems of points in space.” *Philosophical Magazine*, **2**(6), 559–572.

Affiliation:

Giovanni Merola
British University Vietnam
Ecopark, Hung Yen
Vietnam
E-mail: merolagio@gmail.com